Timestamp Specification

Signal Timestamp vs Snapshot Timestamp

All valid values · Edge cases · API usage · Developer guidance

v1.0 | May 2026 | Internal Engineering + SDK Reference

1. The Fundamental Difference

1.1 One-Line Definitions

Timestamp Type	One-Line Definition	Triggered By	Trading Significance
Signal Timestamp	The exact nanosecond a specific market condition BECAME TRUE	A market event — a condition crossing a threshold for the first time	HIGH — marks the birth of an actionable setup
Snapshot Timestamp	The exact nanosecond a periodic reading of an indicator was RECORDED	A timer or bar close — regardless of whether the value changed	LOW — reflects current state, not a new event

1.2 The Smoke Alarm Analogy

Think of it this way:

Scenario	Signal Timestamp	Snapshot Timestamp
Fire alarm system	The moment the alarm FIRES — smoke concentration crossed the threshold. This is an event. It did not exist a millisecond before.	The smoke detector reading the air every 10 seconds and logging the concentration. 9 out of 10 readings may be identical — no alarm yet.
Order flow equivalent	The moment a Stacked Imbalance FORMS — 3rd consecutive imbalance detected at 14:32:07.483291000. This setup did not exist at 14:32:07.483290999.	The Delta value being streamed every 100ms: 14:32:07.000 delta=+142, 14:32:07.100 delta=+145, 14:32:07.200 delta=+148 — incrementally changing.
Developer implication	Subscribe and wait. A signal stream is quiet most of the time. When it fires, ACT — something significant just happened.	Always has a value. Use for dashboards, charts, position sizing calculations. Do not treat a new snapshot as a new setup.

1.3 Side-by-Side Architecture

In the engine, these two timestamp types are produced by completely different subsystems:

Property	Signal Timestamp (ts)	Snapshot Timestamp (ts)
Subsystem	Signal Detection Engine — event-driven	Metric Streaming Engine — timer-driven
Trigger	Condition threshold crossed for first	Timer fires (100ms, 5s) OR bar

	time	closes
Frequency	Sparse — minutes or hours between signals	Dense — tens to hundreds per minute
Deduplication	YES — same signal not re-emitted until reset	NO — snapshots repeat even if value unchanged
Value change required?	YES — signal only fires when something NEW happens	NO — snapshot emits even if value is identical
Use in API query	Filter: 'show me signals BETWEEN these two times'	Filter: 'show me the indicator VALUE AT this time'
Persisted in DB?	YES — every signal permanently stored with full context	NO — only latest value in Redis; historical via OHLCV bars
Examples	Pulse signal, Turns signal, Stacked imbalance detection, Single print	Live delta value, current VWAP level, current session POC
WebSocket behavior	Fires once per event. Quiet between events.	Fires on every timer tick. Continuous stream.
Nanosecond precision?	YES — must be exact for proper bar attribution	YES — but less critical; nearest 100ms is acceptable

2. Signal Timestamp — Complete Specification

2.1 What a Signal Timestamp Records

A signal record contains not ONE timestamp but FOUR distinct timestamps, each with a different meaning. Developers must understand which one to use for which purpose.

Timestamp Field	What It Records	Precision	Use Case
detection_ts	The nanosecond the engine's signal detection algorithm determined the condition was met. This is computed AFTER the triggering bar closes (or tick arrives).	Nanoseconds (int64)	Primary signal timestamp. Use this for 'when did this signal fire?' Use in backtesting as the action timestamp.
bar_open_ts	The nanosecond the BAR that triggered the signal OPENED. This is always earlier than detection_ts.	Nanoseconds (int64)	Bar attribution. Use to identify WHICH bar caused the signal. Use when aligning signal with chart data.
bar_close_ts	The nanosecond the triggering bar CLOSED and signal evaluation ran. For bar-close signals this equals or is very close to detection_ts.	Nanoseconds (int64)	Execution timing. Tells developer: 'you could have entered at the NEXT bar open after this time.'
exchange_ts	The exchange-assigned timestamp of the LAST TICK that completed the triggering condition. This is the most precise market-event timestamp.	Nanoseconds (int64)	Regulatory / audit use. Matches exchange records. Use when reporting or reconciling with broker data.

2.2 Timeline Relationship Between Signal Timestamps

These four timestamps always follow this strict ordering:

```
exchange_ts  <=  bar_open_ts  <  bar_close_ts  <=  detection_ts
```

Concrete example — a Stacked Imbalance signal on ES 5-minute chart:

Timestamp Field	Value (Example)	What Happened at This Moment
bar_open_ts	2026-05-30 09:30:00.000000000	The 9:30 AM bar opened. Ticks began flowing in.

exchange_ts	2026-05-30 09:34:47.291847362	The 3rd consecutive imbalance tick arrived at the exchange. This completed the stacking condition.
bar_close_ts	2026-05-30 09:35:00.000000000	The 5-minute bar closed. Signal evaluation ran on the completed bar data.
detection_ts	2026-05-30 09:35:00.000003847	The signal detection algorithm finished processing and recorded the signal. 3.8 microseconds after bar close.

For most developers, detection_ts is the timestamp to use. It is the moment you would have KNOWN about the signal and been able to act.

2.3 Valid Values a User Can Enter for Signal Timestamp

When a user queries the API for historical signals or configures time-based filters, the following timestamp formats and values are accepted:

2.3.1 Accepted Timestamp Formats

Format	Example	Notes
Unix nanoseconds (int64)	1748599800000000000	Preferred internal format. 19 digits. No ambiguity. Most precise.
Unix milliseconds (int64)	1748599800000	13 digits. Accepted; internally converted to nanoseconds (×1,000,000).
Unix seconds (int64 or float64)	1748599800 or 1748599800.291	10 digits integer OR float with decimal fraction. Fractional seconds accepted.
ISO 8601 UTC string	2026-05-30T09:35:00.000000000Z	Human-readable. Z suffix required (UTC). Nanoseconds in fraction optional.
ISO 8601 with offset	2026-05-30T04:35:00-05:00	Offset from UTC accepted. Internally converted to UTC nanoseconds.
Date-only string (00:00:00 UTC)	2026-05-30	Expands to 2026-05-30T00:00:00.000000000Z. Useful for daily queries.
Relative shorthand	now, now-1d, now-1w, now-1h, session_open, session_close	Convenience shortcuts (see §2.3.2)

2.3.2 Relative Timestamp Shorthand Values

These string values are resolved server-side at query time to an absolute UTC nanosecond timestamp:

Shorthand Value	Resolves To	Use Case
now	Current server time in UTC nanoseconds	Get signals up to the present moment
now-Ns	N seconds ago (e.g. now-30s = 30 seconds ago)	Recent signal window
now-Nm	N minutes ago (e.g. now-15m = 15 minutes ago)	Intraday lookback
now-Nh	N hours ago (e.g. now-4h = 4 hours ago)	Session lookback
now-Nd	N days ago (e.g. now-5d = 5 days ago)	Multi-day lookback
now-Nw	N weeks ago (e.g. now-2w = 2 weeks ago)	Swing trade lookback
now-NM	N months ago (e.g. now-3M = 3 months ago)	Strategy validation window
now-Ny	N years ago (e.g. now-1y = 1 year ago)	Annual backtest range
session_open	Today's regular trading hours open (instrument-specific, e.g. 09:30:00 ET for ES)	Current session signals only
session_close	Today's regular trading hours close (instrument-specific, e.g. 16:00:00 ET for ES)	Signals before close
prev_session_open	Previous trading day's session open	Yesterday's signals
prev_session_close	Previous trading day's session close	Yesterday's close context
week_open	Monday's session open for the current week	Week-to-date signals
month_open	First trading day of current calendar month, session open	Month-to-date signals
year_open	First trading day of current calendar year, session open	Year-to-date signals
epoch	1970-01-01T00:00:00.000000000Z (Unix epoch zero)	Fetch ALL available history from the beginning

2.3.3 Absolute Timestamp Valid Range

Boundary	Value	Behaviour if Exceeded
Minimum valid ts	2010-01-01T00:00:00Z (Unix ns: 1262304000000000000)	API returns HTTP 400: timestamp_too_early. No data before this date.

Maximum valid ts	now + 1 second (server clock tolerance)	API returns HTTP 400: timestamp_in_future. Prevents nonsensical future queries.
Maximum range (Basic tier)	30 days between from_ts and to_ts	HTTP 403: range_exceeds_tier_limit
Maximum range (Standard)	365 days between from_ts and to_ts	HTTP 403: range_exceeds_tier_limit
Maximum range (Professional)	5 years between from_ts and to_ts	HTTP 403: range_exceeds_tier_limit
Maximum range (Enterprise)	Full history — no cap	No restriction
Null / omitted from_ts	Defaults to session_open (today)	No error — sensible default applied
Null / omitted to_ts	Defaults to now	No error — sensible default applied

2.3.4 Valid and Invalid Timestamp Examples

✓ *1748599800000000000 — Valid: Unix nanoseconds, 19 digits, within range*

✓ *2026-05-30T09:35:00Z — Valid: ISO 8601 UTC, Z suffix*

✓ *2026-05-30T04:35:00-05:00 — Valid: ISO with offset (resolves to same UTC)*

✓ *now-4h — Valid: relative shorthand*

✓ *session_open — Valid: session-relative shorthand*

✓ *1748599800 — Valid: Unix seconds (10 digits)*

✓ *1748599800.291 — Valid: Unix seconds with fractional part*

✓ *2026-05-30 — Valid: date-only (expands to midnight UTC)*

✓ *epoch — Valid: fetch all history*

✗ *1748599800000 — Ambiguous: could be milliseconds OR a very old nanosecond timestamp. API interprets by digit count: 13 digits = milliseconds.*

✗ *2026-05-30T09:35:00 — Invalid: missing timezone suffix. UTC is not assumed. Returns HTTP 400: missing_timezone.*

✗ *05/30/2026 09:35:00 — Invalid: US date format not supported. Returns HTTP 400: invalid_timestamp_format.*

✗ *2026-13-45T99:99:99Z — Invalid: impossible date values. Returns HTTP 400: invalid_timestamp_value.*

✗ *99999999999999999999 — Invalid: exceeds int64 range. Returns HTTP 400: timestamp_overflow.*

✗ *now+1d — Invalid: future timestamps not allowed. Returns HTTP 400: timestamp_in_future.*

✗ *0 — Invalid: Unix epoch zero predates all valid data. Returns HTTP 400: timestamp_too_early.*

⚠️ *Always prefer Unix nanoseconds (int64) in production code. String formats require server-side parsing and add ~0.2ms latency to query resolution.*

2.4 Signal Timestamp in Different API Contexts

API Context	Parameter Name	Which Signal Timestamp Field?	Example
REST historical query — start	from_ts	Filters on detection_ts >= from_ts	GET /v1/signals/pulse/ES?from_ts=2026-05-30T09:30:00Z
REST historical query — end	to_ts	Filters on detection_ts <= to_ts	GET /v1/signals/pulse/ES?to_ts=now
REST — align to bar	bar_ts	Filters on bar_close_ts == bar_ts	Fetch the signal from a specific bar close
REST — align to exchange event	exchange_ts	Filters on exchange_ts >= from_ts	Regulatory / audit queries only
REST — sort order	order_by	Sort by detection_ts ASC or DESC	order_by=detection_ts&direction=desc
WebSocket — replay from past	since	Replays all signals with detection_ts >= since on connect	SUBSCRIBE pulse:ES?since=now-1h
WebSocket — live only	(no since param)	Receives only signals with detection_ts > connection time	SUBSCRIBE pulse:ES
OFE-Script backtest — start	from_date	Resolves to detection_ts >= date 00:00:00 UTC	from_date=2025-01-01
Backtest — end	to_date	Resolves to detection_ts <= date 23:59:59.999 UTC	to_date=2025-12-31
Alert — cooldown window	cooldown_seconds	Suppresses re-alerts for N seconds after detection_ts	cooldown_seconds=300 = no repeat within 5 min

2.5 Signal-Specific Timestamp Subtleties

Different signal types have important nuances in how their timestamps should be interpreted:

Signal Type	detection_ts Behavior	Important Nuance
Stacked Imbalance	Set when the 3rd (or Nth) consecutive imbalance is detected — on bar close	Zone_id persists. Subsequent bars that EXTEND the zone do NOT generate a new detection_ts. Use zone_created_ts vs zone_last_updated_ts to distinguish.
Orderflows Pulse	Set at bar close when composite score crosses the configured threshold for first	Pulse can oscillate: if score drops below threshold and rises again on the next bar, a new signal fires with a new detection_ts.

	time	
Orderflows Turns	Set at the TICK level when the turning point pattern completes — NOT bar close	exchange_ts and detection_ts may differ by microseconds only. Most precise signal in the system.
Orderflows Ratio (Big Buyer/Seller)	Set at bar close when bid/ask ratio at extreme price exceeds threshold	One signal per bar maximum. If multiple bars meet criteria consecutively, each gets its own detection_ts.
Single Print / Last Buyer/Seller	Set at bar close when zero/near-zero volume detected at extreme	magnet_active=true persists until price revisits the level. detection_ts is the birth; magnet_deactivation_ts records when the magnet was resolved.
Delta Surge	Set when surge_magnitude crosses threshold — updated every 100ms during the bar	detection_ts is the FIRST moment the surge threshold was crossed, not the peak. surge_peak_ts records when the surge was at its maximum.
CVD Divergence	Set when divergence condition has been true for N consecutive bars	detection_ts is when the Nth bar confirmed the pattern. divergence_start_ts is when the pattern first began.
Volume Profile Shape	Set at end of session when shape is classified	For live sessions, shape_ts updates on each bar close as the profile builds and shape may change.
VWAP Reaction Signal	Set when price comes within configured ticks of VWAP and order flow confirms	signal_type=LONG_SETUP vs SHORT_SETUP. detection_ts is the tick-level moment price touched VWAP, not bar close.
Unfinished Business / Single Print magnet	Initial: bar close of the creating bar. Resolved: bar close when price revisits.	Two timestamps exist: creation_ts (when the magnet was created) and resolution_ts (when it was resolved). Query unresolved magnets with resolution_ts=null.

3. Snapshot Timestamp — Complete Specification

3.1 What a Snapshot Timestamp Records

A snapshot is a periodic read of a continuously-computed indicator value. Unlike signals, snapshots have no concept of 'first occurrence' — they simply record the current state at a point in time.

Snapshot Field	What It Records	Precision	Reset Behavior
ts	The server clock time when this snapshot was generated and dispatched to subscribers	Nanoseconds (int64)	Never resets — monotonically increasing wall clock
value_as_of_ts	The exchange timestamp of the last tick that caused the indicator to change. May be older than ts by up to 100ms.	Nanoseconds (int64)	Updates only when a new tick arrives
bar_ts	The open timestamp of the current (building) bar this snapshot belongs to	Nanoseconds (int64)	Resets on each new bar open

3.2 Snapshot Frequency by Indicator

Indicator	Snapshot Frequency	Trigger	ts Meaning
Live Delta	Every 100ms during market hours	Timer	Wall clock time of emission
Cumulative Delta (CVD)	Every 100ms during market hours	Timer	Wall clock time of emission
VWAP (all types)	Every 5 seconds OR on every bar close	Timer + bar close	Wall clock time of emission
Volume Profile (live)	On every bar close	Bar close	Equals bar_close_ts of completing bar
Volume Profile (session)	On every bar close	Bar close	Equals bar_close_ts of completing bar
VWAP Deviation Bands	Every 5 seconds	Timer	Wall clock time of emission
Footprint bar (complete)	On bar close only	Bar close	Equals bar_close_ts — not wall clock
Open POC (building bar)	Every 100ms during bar construction	Timer	Wall clock time of emission
POC Migration tracking	On every bar close	Bar close	Equals bar_close_ts

Session High / Low	Every tick that sets new extreme	Tick event	Equals exchange_ts of the new extreme tick

3.3 Valid Values for Snapshot Timestamp Queries

When querying historical snapshots, the timestamp logic differs slightly from signal queries:

Query Type	Parameter	Behavior	Example
Point-in-time lookup	at_ts	Returns the indicator value AT or immediately before the given timestamp. Finds nearest snapshot <= at_ts.	GET /v1/delta/ES/live?at_ts=2026-05-30T10:15:00Z
Range query	from_ts + to_ts	Returns all snapshots where ts >= from_ts AND ts <= to_ts	GET /v1/vwap/ES?from_ts=session_open&to_ts=now
Resolution control	resolution	Downsample snapshots to a lower frequency to reduce payload size	resolution=1m returns one snapshot per minute (last value in period)
Bar-aligned query	bar_close_only=true	Returns only snapshots taken at bar close — eliminates intra-bar noise	GET /v1/profile/ES/session?bar_close_only=true
Latest snapshot only	latest=true	Returns single most recent snapshot — no from/to needed	GET /v1/vwap/ES?latest=true

3.4 Snapshot Timestamp Accepted Formats

Snapshot timestamp queries accept the same formats as signal timestamps (see §2.3.1 and §2.3.2). The same rules apply:

- Unix nanoseconds (int64) — preferred
- Unix milliseconds / seconds — accepted, auto-detected by digit count
- ISO 8601 UTC string with Z suffix — accepted
- ISO 8601 with timezone offset — accepted
- Date-only string — expands to 00:00:00.000 UTC
- All relative shorthands (now, now-Nm, session_open, etc.) — accepted

For snapshot point-in-time queries using at_ts, the API returns the LAST snapshot taken at or before that time — not an interpolated value. If no snapshot exists before at_ts, the API returns HTTP 404: no_snapshot_before_timestamp.

4. Edge Cases & Special Scenarios

4.1 Pre-Market & After-Hours

Scenario	Signal Timestamp Behavior	Snapshot Timestamp Behavior
Pre-market (before 09:30 ET for ES)	Signals CAN fire during pre-market if instrument trades. detection_ts will have pre-market time. Filter with session_filter=rth_only to exclude.	Snapshots continue streaming if market is open. Lower volume means less frequent value changes.
After-hours (after 16:00 ET for ES)	Same as pre-market — signals fire if the instrument is trading. session_filter=rth_only excludes these.	VWAP snapshot continues using the RTH session VWAP (no reset until next session open).
Weekend / market closed	No signals fire. Signal stream is completely silent.	No snapshots emitted. WebSocket connection stays alive with heartbeat only.
Holiday	No signals fire. No snapshots. Same as weekend.	Same — no snapshots. Heartbeat only.
Instrument halt (circuit breaker)	Signals stop. detection_ts of last signal before halt remains in history.	Snapshots stop. Last snapshot value frozen. halt_active=true flag added to subsequent heartbeats.

4.2 Out-of-Order Ticks

Exchanges occasionally deliver ticks out of sequence. The engine handles this and it affects timestamps:

- The engine uses exchange_ts (exchange sequence number) for ordering, NOT server receipt time
- If a tick arrives late with an exchange_ts earlier than the last processed tick, it is inserted retroactively into the bar's price level data
- If the retroactive tick changes a signal condition, the signal is RE-EVALUATED but the detection_ts is NOT backdated — it records when the engine actually determined the signal
- A late_tick_adjustment=true flag is added to any signal that was influenced by a retroactively corrected tick
- Developers should not re-order signals by exchange_ts — always use detection_ts for strategy logic

4.3 Bar Boundary Signals

Signals that fire exactly at bar close need special handling:

Scenario	Timestamps	Developer Action
Signal fires AT bar close	bar_close_ts ≈ detection_ts	Entry should be at NEXT bar open. Do

	(within microseconds). The signal is based on the completed bar.	not enter on the same bar that generated the signal.
Signal fires MID-BAR (tick-level signals)	exchange_ts < bar_close_ts. detection_ts is set to the tick time, not bar close.	Turns and VWAP Reaction signals can fire mid-bar. Entry may be possible on the same bar.
Two signals on same bar	Same bar_open_ts and bar_close_ts. Different detection_ts (microseconds apart).	Treat as concurrent signals. Both valid. Apply your own priority logic.
Signal fires on bar that is then revised (rare)	Original detection_ts kept. A revision_ts field is added with revised signal values.	Check revision_ts field. If present, use the revised signal data.

4.4 Session Rollover

At session boundaries, timestamps require careful handling:

- Session rollover time varies by instrument: ES = 16:00 ET (17:00 ET for Globex); forex = 17:00 ET; crypto = 00:00 UTC
- CVD (Cumulative Delta) resets to zero at session_open. The final CVD of the old session and 0 of the new session share a rollover_ts timestamp.
- VWAP resets to session open price at session_open. The snapshot immediately after session_open will show daily_vwap = session_open_price.
- Volume Profile resets at session_open. A profile_reset_ts field marks when the profile cleared and restarted.
- Signals that straddle a session boundary (e.g., stacked imbalance zone that formed in the prior session): zone persists but zone_session_origin records which session created it.

⚠ *Do NOT query from_ts=prev_session_close and to_ts=session_open expecting continuity. A gap exists between sessions. Use prev_session_open to session_close for prior session data, then session_open to now for current.*

4.5 Daylight Saving Time

All timestamps in the API are stored and transmitted in UTC nanoseconds. DST transitions are handled automatically:

- The API NEVER uses 'Eastern Time' or 'Central Time' internally — everything is UTC
- Session open/close shorthands (session_open, session_close) are resolved to UTC accounting for DST at query time
- In the US: ES session_open = 14:30:00 UTC (winter) or 13:30:00 UTC (summer) — resolved correctly by server
- Developers using relative shorthands never need to worry about DST — the server handles it
- Developers using absolute UTC timestamps are immune to DST by definition

✓ *Always store and transmit timestamps as Unix nanoseconds in production. They are UTC by definition and DST-immune.*

4.6 Clock Synchronization Tolerance

Source	Maximum Clock Drift Tolerated	Action if Exceeded
Engine server vs exchange	50ms	Alert to ops team. Tick classification may be affected.
API server vs engine	10ms	ts field may be up to 10ms after actual value change
Client vs API server	5 minutes	JWT tokens with 15-min expiry tolerate up to 5-min client clock skew
WebSocket heartbeat timeout	30 seconds	Connection closed if no heartbeat response in 30s. Reconnect.

5. Quick Reference — Developer Cheat Sheet

5.1 'Which timestamp should I use?' Decision Guide

I want to...	Use this timestamp	Parameter / Field
Get all Pulse signals today	detection_ts >= session_open	from_ts=session_open
Get signals in a specific hour	detection_ts range	from_ts=2026-05-30T10:00:00Z&to_ts=2026-05-30T11:00:00Z
Know what bar caused the signal	bar_close_ts or bar_open_ts	bar_close_ts field in response
Align signal with my chart data	bar_open_ts	Match to your bar's open time
Find entry timing after signal	bar_close_ts + 1 bar	Enter at open of bar AFTER bar_close_ts
Match to broker trade record	exchange_ts	exchange_ts field for regulatory queries
Get current VWAP value	Snapshot: latest=true	GET /v1/vwap/ES?latest=true
Get VWAP at 10:00 AM today	Snapshot: at_ts	GET /v1/vwap/ES?at_ts=2026-05-30T14:00:00Z
Watch live delta building	Snapshot stream: ts	SUBSCRIBE delta:live:ES — use ts field
Run backtest last 6 months	detection_ts range	from_ts=now-6M&to_ts=now
Get only RTH session signals	Session filter shorthand	from_ts=session_open&to_ts=session_close
Get last 5 Pulse signals	detection_ts DESC + limit	order_by=detection_ts&direction=desc&limit=5
Check if stacked zone still active	zone_id + active status	GET /v1/zones/ES/{zone_id}?field=is_active
Find unresolved Single Print magnets	resolution_ts is null	GET /v1/signals/single_prints/ES?resolved=false

5.2 Timestamp Format Quick Reference

Format	Example	Digits / Pattern	Use When
Unix nanoseconds	1748599800000000000	19 digits	Production code — fastest, unambiguous
Unix milliseconds	1748599800000	13 digits	JavaScript Date.now() compatibility

Unix seconds	1748599800	10 digits	Simple scripts
ISO 8601 UTC	2026-05-30T09:35:00Z	Z suffix required	Human-readable configs
ISO 8601 + nanoseconds	2026-05-30T09:35:00.000003847Z	Up to 9 decimal places	Maximum precision in configs
ISO 8601 + offset	2026-05-30T04:35:00-05:00	±HH:MM suffix	When working in local time
Date only	2026-05-30	YYYY-MM-DD	Daily boundary queries
now	now	Keyword	Real-time end of range
now-Xunit	now-4h, now-30m, now-1d	Relative expression	Rolling windows
session_open	session_open	Keyword	Current session start
epoch	epoch	Keyword	Fetch all history

5.3 HTTP Error Codes for Timestamp Errors

HTTP Status	Error Code	Cause	Fix
400	invalid_timestamp_format	Unrecognised format (e.g. MM/DD/YYYY)	Use ISO 8601 or Unix int
400	missing_timezone	ISO string without Z or offset	Add Z suffix for UTC
400	invalid_timestamp_value	Impossible date (month 13, hour 99)	Fix the date value
400	timestamp_overflow	Exceeds int64 range (>9223372036854775807)	Check for unit confusion (ns vs µs)
400	timestamp_too_early	Before 2010-01-01 or Unix epoch zero	Use epoch keyword for maximum history
400	timestamp_in_future	to_ts > now + 1 second	Remove future timestamps; use now
400	from_ts_after_to_ts	from_ts >= to_ts	Swap the values; from < to
403	range_exceeds_tier_limit	Date range wider than subscription allows	Narrow the range or upgrade tier
404	no_snapshot_before_timestamp	at_ts query: no snapshot exists before that time	Use a later at_ts or check market hours
422	ambiguous_timestamp	10-digit value: treated as seconds, not ms or ns	Use 13 or 19-digit form to be explicit

END OF DOCUMENT — Timestamp Specification v1.0

Covers: 4 signal timestamp fields · all accepted formats · 17 relative shorthands · snapshot vs signal difference · edge cases · quick reference